



AI with Rav

Learn AI the way you actually think.



DAY 14 of 30

MACHINE LEARNING

PCA — casting the perfect shadow

WHAT YOU'LL LEARN TODAY

- › Why too many columns is a real problem
- › PCA = casting a shadow on the wall
- › The best angle = where the data spreads most
- › Bad shadow vs good shadow
- › How much shape you keep (variance)



PCA — casting the perfect shadow

In One Line: Imagine shining a light on a 3D object so it casts a flat shadow on the wall. If you turn the object to the best angle, the flat shadow STILL shows what it is. PCA is that trick — it squishes many columns down to 2 while keeping the shape.

Start here: too many columns to even see

Every dataset is a table — rows are people, columns are things you measured. With 2 columns (say height and weight), you can plot them and see the pattern with your eyes. Easy.

But real data often has **hundreds of columns**: age, income, heart-rate, sleep, steps... 100 of them. You **cannot draw a 100-dimension picture** — your eyes only do 2 or 3. So how do you see the shape of data with too many columns? That's the problem **PCA (Principal Component Analysis)** solves.

The problem: too many columns

The problem: too many columns to picture

age	income	height	weight	steps	sleep	heart	...	col 100
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.

100 columns — impossible to SEE the pattern by eye

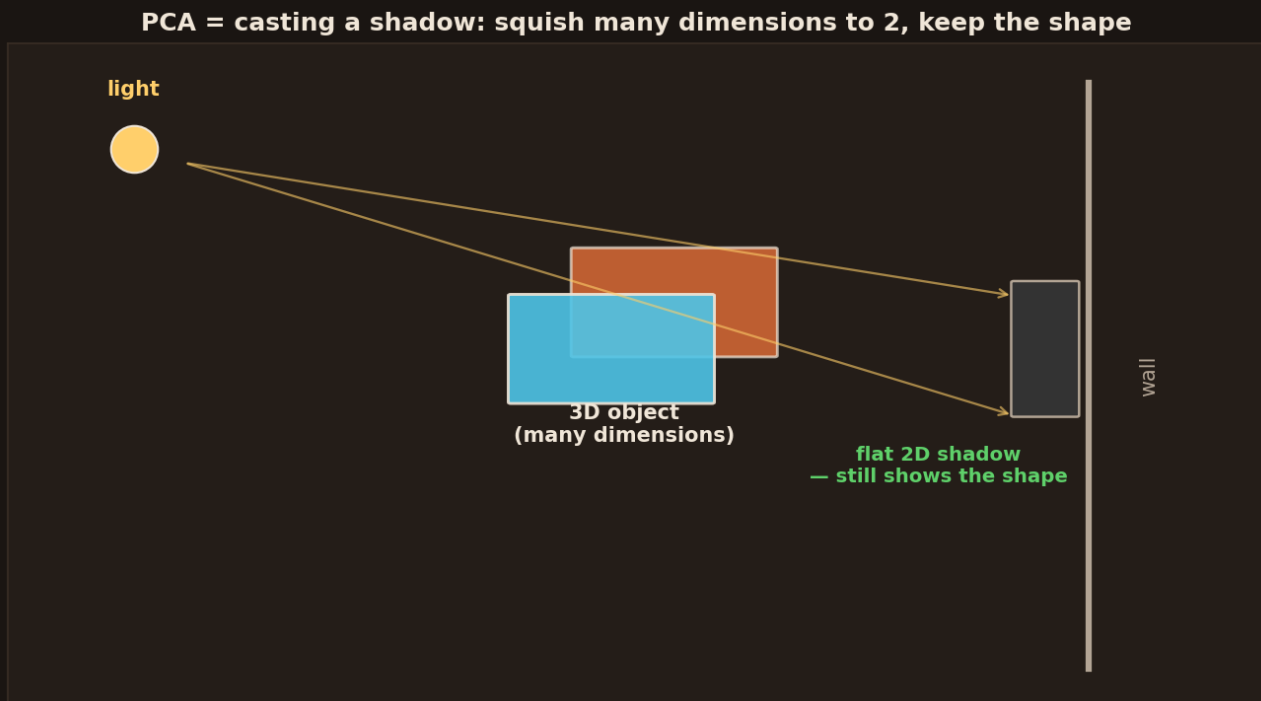
A table with 100 columns — age, income, height, weight, and on and on. You simply can't picture 100 dimensions at once. The pattern is hidden in there, but invisible to the eye.

- 2 columns → you can plot a simple graph and see it.
- 3 columns → still just about possible (a 3D plot).
- **100 columns** → impossible to picture. The pattern is trapped, unseen.



We need a way to **shrink** 100 columns down to just 2 or 3 — few enough to plot — WITHOUT losing the important shape hiding inside. That squishing-while-keeping-the-shape is exactly PCA's job.

PCA = casting a shadow



Shine a light on a 3D object and it throws a flat 2D shadow on the wall. The shadow has fewer dimensions — but it still shows the object's shape. PCA casts that shadow for data.

Here's the beautiful idea. Take a **3D object** — say a teapot. It has depth, width, height (3 dimensions). Shine a light on it and its **shadow on the wall** is flat — only 2 dimensions. Yet you can still tell it's a teapot!

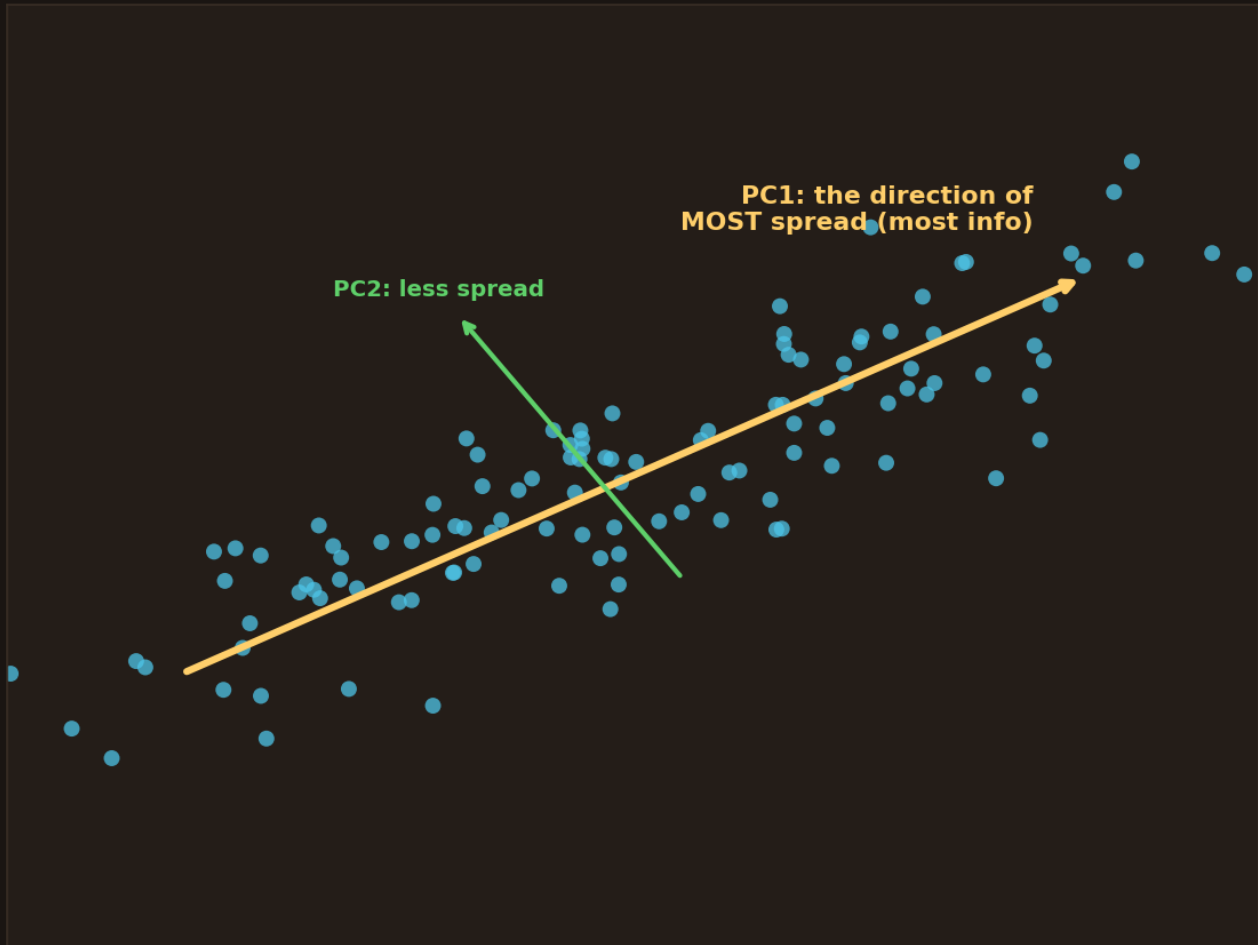
- The object = your data with **many** columns.
- The shadow on the wall = your data squished to **2** columns.
- A good shadow still shows the **shape** — that's the whole trick.

PCA "casts a shadow" of your high-dimension data onto a flat 2D surface you can actually plot. Fewer columns, but the shape survives. The only question left is: *from which angle should we cast the shadow?* Because the angle changes everything.

The best angle: where the data spreads most



PCA finds the BEST angle: where the data spreads out most



The data cloud spreads out most along one diagonal direction (the long yellow arrow, PC1). PCA finds that direction of biggest spread — that's where the most information lives.

Turn a teapot the wrong way and its shadow is a thin useless blob. Turn it right and the shadow clearly shows the spout and handle. **The angle matters.** So how does PCA pick the best angle?

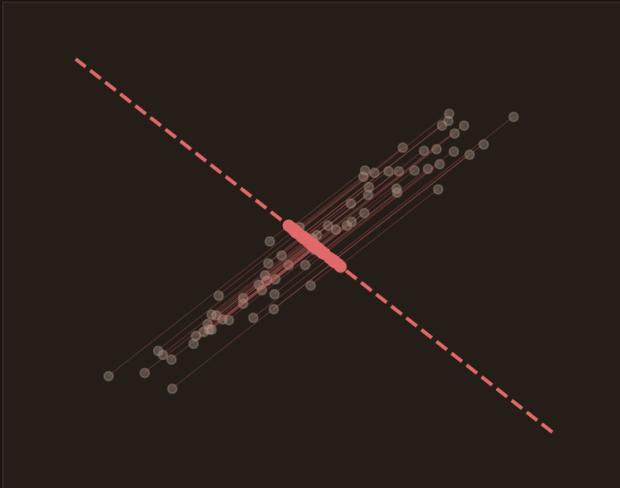
- It looks for the direction where the data **spreads out the most**.
- That direction of biggest spread is called **PC1** (the 1st Principal Component) — it holds the **most information**.
- The next-best direction (at a right angle to it) is **PC2**, holding the second-most.

Why "most spread = most information"? Because if everyone has almost the same value on some direction, that direction tells you **nothing** (no spread, no info). The directions where people **differ the most** are the ones worth keeping. PCA ranks directions by spread and keeps the top ones.

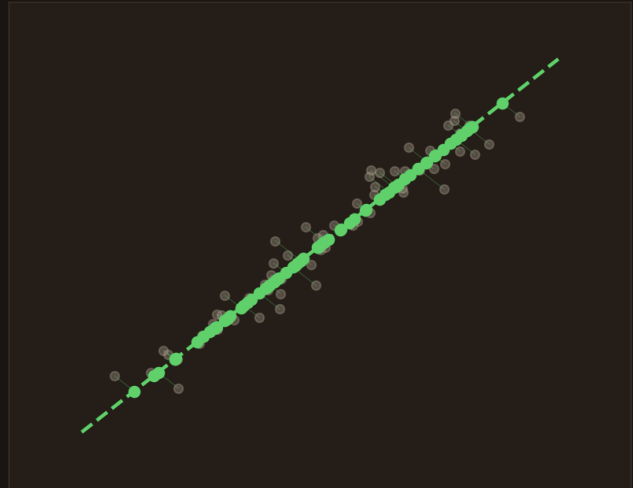
Bad shadow vs good shadow



**BAD angle: shadows pile up —
lose the shape**



**GOOD angle: shadows spread out —
keep the shape**



Left: cast the shadow at a BAD angle (across the spread) and all the points pile into a tiny clump — you lose the shape. Right: cast at the GOOD angle (along the spread) and the points stay spread out — the shape survives.

Same data, two different "walls" to cast onto:

- **Bad angle** → you flatten the data *across* its spread, so all the points **pile into one clump**. Everyone looks the same — the shape is destroyed.
- **Good angle** (PC1) → you flatten *along* its spread, so the points **stay spread out** in a line. The differences survive — the shape is kept.

That's the entire art of PCA: **cast the shadow at the angle that keeps the points spread out**. The left picture lost everything; the right kept the shape. PCA always finds the "right" angle for you automatically.

The "formula" — but really simple

No scary maths needed to *get* it. PCA measures spread with a number called **variance** (just "how spread out is it?").

PC1 = the direction with the MOST variance (spread). PC2 = the next most, at a right angle. Keep the top few, drop the rest.

What each word means, in shadow terms:

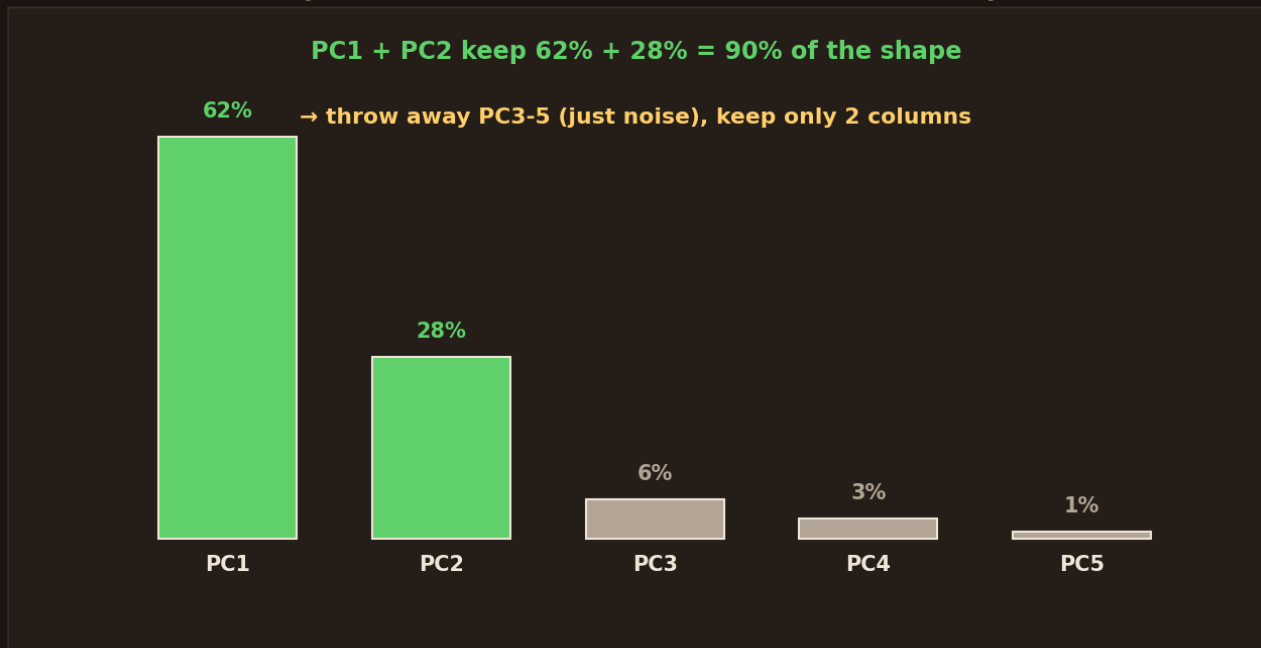
- **Variance** → how spread out the data is along a direction. More spread = more information kept.
- **Principal Component** → just a fancy name for "a direction we cast the shadow along," ranked by how much spread it captures.
- **PCA** → line up all the directions by spread, keep the top 2 or 3, throw the rest away.



So the whole engine is: **find the directions of biggest spread, keep a few, drop the rest**. The kept directions hold the shape; the dropped ones were mostly noise. No calculus needed to use it — just "keep the directions that spread the most."

How much shape did we keep?

Keep the few directions that hold most of the shape



Each component captures a slice of the shape. Here PC1 holds 62% and PC2 holds 28% — together 90%. PC3-5 hold almost nothing (noise), so we drop them and keep just 2 columns.

PCA even tells you **how much of the shape you kept**. Each direction captures a percentage:

- **PC1** captures 62% of the shape, **PC2** captures 28% → together **90%**.
- **PC3, PC4, PC5** capture 6%, 3%, 1% — almost nothing (that's the noise).
- So keep **PC1 + PC2** (2 columns) and you've kept **90% of the shape** while throwing away 98 columns!

This is the payoff: from 100 columns down to 2, keeping 90% of the real shape. You pick how many to keep by asking "how much shape do I want?" — usually keep enough components to hold ~90-95%.

Try it — a few lines



```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)          # squish down to 2 columns
X_small = pca.fit_transform(X)    # X had 100 columns; X_small has 2

print(pca.explained_variance_ratio_) # e.g. [0.62, 0.28] -> 90% kept
# now you can plot X_small on a normal 2D graph and SEE the groups!
```

`n\_components=2` says "give me 2 columns." `explained\_variance\_ratio\_` tells you the % of shape each kept — add them up to see how much you saved. After PCA, you can finally **plot and see** data that had too many columns.

Where it shines and where it slips

- **See the unseeable** → squish 100+ columns to 2 so you can plot and eyeball the groups. Great before clustering (pairs beautifully with K-Means).
- **Speeds things up + removes noise** → fewer columns = faster models, and the dropped directions were mostly noise anyway.
- **New columns aren't "real"** → PC1 and PC2 are *mixes* of the originals, so you lose easy names like "age" or "income." Harder to explain to your boss.
- **Only catches straight-line spread** → PCA looks for spread along straight directions; if the real shape is curvy or twisted, it can miss it (fancier methods like t-SNE/UMAP handle that).
- **Scale your columns first** → like KNN and K-Means, PCA measures spread, so a big-numbered column (salary) would dominate. Always scale first.

The one thing to remember: PCA is your tool to **shrink too many columns down to a few you can see** — casting the best shadow that keeps the shape. Just scale your columns first, and remember the new columns are mixes, not the originals.

Recap — the 20-second version

- PCA **squishes many columns down to 2 or 3** so you can finally see the data.
- It's like **casting a shadow** — fewer dimensions, but the shape survives.
- It picks the **best angle** = the direction where the data **spreads most** (most info).
- A **good shadow keeps the spread**; a bad one piles everything into a clump.
- It tells you **how much shape you kept** (variance) — usually aim for ~90%.



Next up — Video 15: Overfitting & Underfitting. You've now met a whole toolbox of algorithms. But every single one can fail the same two ways — memorising too much, or learning too little. Next we learn the most important lesson in all of ML: how to make a model that actually works on NEW data. See you Day 15.

